

Computer Vision - HW1 Camera Calibration

Group 18

312552026 蔡濟謙, 313552049 鄭博涵, 110550035 陳奎元

1. Introduction

Camera calibration is the process of determining the internal characteristics (intrinsic parameters), lens distortion, and the camera's spatial relationship with the surrounding environment (extrinsic parameters).

This process allows us to understand how a 3D scene is projected onto the 2D image captured by the camera, ensuring accurate measurements and mapping between the real world and the captured image.

Camera calibration is a critical process in a wide range of applications, particularly those involving image analysis, 3D reconstruction, and measurements.

In this homework, we implemented camera calibration from scratch, deriving the camera's intrinsic matrix and the extrinsic matrices for each image of the chessboard without considering the lens distortion.

2. Implement Procedure

The coordinates of all points on the chessboard are generated using `"cv2.findChessboardCorners()"`. The coordinates on the image plane are defined as (u, v) , corresponding to (x, y) on the object plane. The corners of the chessboard on the object plane are defined within the same coordinate system.

- Use the points in each images to find homography matrix

From the formula in the reference, the homography matrix H can be calculated using the object plane. Substituting $M=[x, y, 1]$ results in L .

$$\begin{bmatrix} \tilde{M}^T & 0^T & -u\tilde{M}^T \\ 0^T & \tilde{M}^T & -v\tilde{M}^T \end{bmatrix} x = Lx = 0$$

$$L = \begin{bmatrix} x & y & 1 & 0 & 0 & 0 & -ux & -uy & -u \\ 0 & 0 & 0 & x & y & 1 & -vx & -vy & -v \end{bmatrix}$$

The answer for H is the eigenvector of $L^T L$ associated with the smallest eigenvalue λ . We use “np.linalg.svd()” to obtain H. After obtaining H, the values need to be normalized by dividing all the elements of the matrix by H[2][2], to ensure the proportions are consistent.

Code snippet:

```
H_list = []
for i in range(len(objpoints)):
    obj = objpoints[i] # Get the object points
    img = imgpoints[i].reshape(-1, 2) # Get the image points
    L = []
    for obj, img in zip(obj, img):
        X, Y, _ = obj
        u, v = img
        L.append([X, Y, 1, 0, 0, 0, -u * X, -u * Y, -u])
        L.append([0, 0, 0, X, Y, 1, -v * X, -v * Y, -v])
    L = np.array(L)
    _, _, Vh = np.linalg.svd(L) # singular value decomposition
    Hi = Vh.T[:, -1].reshape(3, 3)
    Hi = Hi / Hi[2, 2] # normalize
    H_list.append(Hi)
```

- Use Hi to find out the intrinsic matrix K

After obtaining the homography matrix H for each image, the next step is to find the intrinsic matrix. Let B be a symmetric matrix, along with the matrix b.

$$B = K^{-T} K^{-1} = \begin{bmatrix} B_{11} & B_{12} & B_{13} \\ B_{12} & B_{22} & B_{23} \\ B_{13} & B_{23} & B_{33} \end{bmatrix}$$

$$b = [B_{11} \quad B_{12} \quad B_{22} \quad B_{23} \quad B_{33} \quad B_{11}]^T$$

Let the i column vector of H be $h_i = [h_{i1}, h_{i2}, h_{i3}]$. Then, we have $h_i^T B h_j = v_{ij}^T b$ with

$$v_{ij} = [h_{i1}h_{j1}, h_{i1}h_{j2} + h_{i2}h_{j1}, h_{i2}h_{j2}, h_{i3}h_{j1} + h_{i1}h_{j3}, h_{i3}h_{j2} + h_{i2}h_{j3}, h_{i3}h_{j3}]^T$$

Finally, using the following formula, we can calculate the eigenvector of $V^T V$ associated with the smallest eigenvalue to obtain the matrix b .

$$\begin{bmatrix} V_{12}^T \\ (V_{11} - V_{22})^T \end{bmatrix} b = 0$$

After obtaining b , we can retrieve B by using the transpose of the matrix and verify the signs of the vector values to confirm the direction of the vector. Finally, we can use "`np.linalg.cholesky()`" to obtain the intrinsic matrix.

Code snippet:

```
_, _, Vt = np.linalg.svd(V)
b = Vt.T[:, -1]
B = np.array(
    [
        [b[0], b[1], b[3]],
        [b[1], b[2], b[4]],
        [b[3], b[4], b[5]],
    ]
)
eigvalsB = np.linalg.eigvals(B) # eigenvalues
if not np.all(eigvalsB > 0): # check if all eigenvalues are
    positive
    B = -B
G = np.linalg.cholesky(B) # cholesky decomposition
k_inv = G.T
K = np.linalg.inv(k_inv) # intrinsic matrix
```

- Find out the extrinsic matrix of each image.

After obtaining the homography matrix and the intrinsic matrix, we can calculate the translation vector and the rotation vector. The rotation vector can be computed using either of the following functions:

“`scipy.spatial.transform.Rotation.from_matrix(r).as_rotvec()`”

or

“`cv2.Rodrigues(r)`”.

Code snippet:

```
h1, h2, h3 = H.T
s = 1 / np.linalg.norm(np.dot(k_inv, h1)) # scale factor
r1 = s * np.dot(k_inv, h1)
r2 = s * np.dot(k_inv, h2)
r3 = np.cross(r1, r2)
r = np.array([r1, r2, r3]).T

tvec = s * np.dot(k_inv, h3) # translation vector
rvec = scipy.spatial.transform.Rotation.from_matrix(
    r
).as_rotvec() # rotation vector

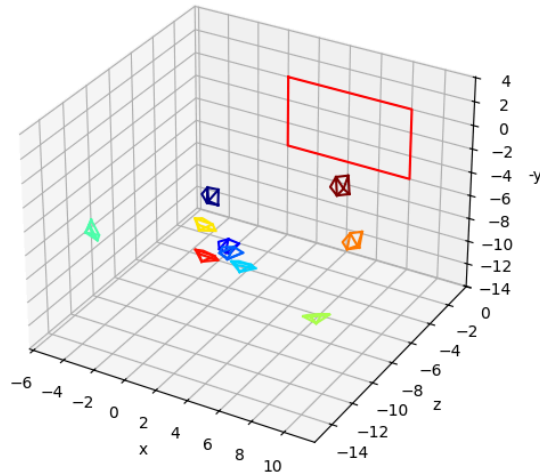
# rvec, _ = cv2.Rodrigues(r)
# rvec = rvec.squeeze()
```

Reference: Z. Zhang, “A Flexible New Technique for Camera Calibration A Flexible New Technique for Camera Calibration,” vol. 10, 1999, Available: <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/tr98-71.pdf>

3. Experimental Result

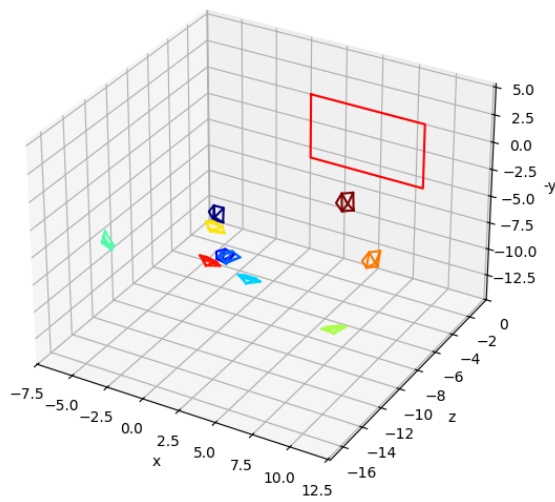
- Teacher provided data:

Extrinsic Parameters Visualization



This is the output from the function provided by cv2.

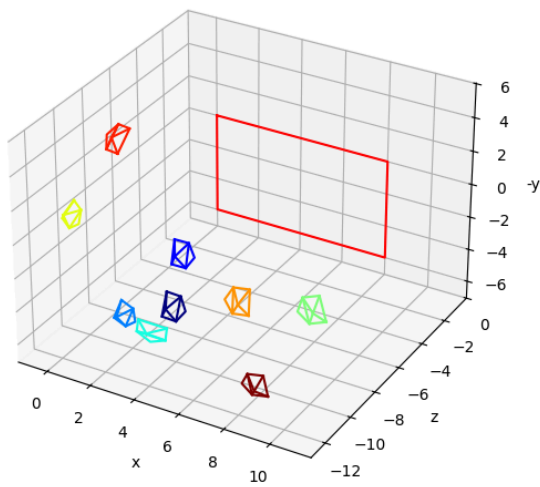
Extrinsic Parameters Visualization



This is the output from our implementation.

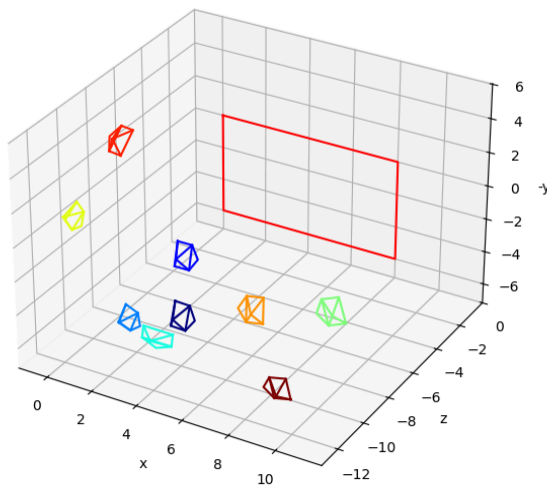
- Our smartphone data:

Extrinsic Parameters Visualization



This is the output from the function provided by cv2.

Extrinsic Parameters Visualization



This is the output from our implementation.

The figures show the positions of different cameras in 3D space and their orientation relative to the chessboard.

Each camera position is calculated based on the extrinsic parameters obtained from the camera calibration process.

From the results, it can be seen that the original cv2 and our implementation yield almost identical outcomes. I think the reason is the camera's position and orientation relative to the chessboard are the same in both cv2 and our implementation.

4. Discussion

Using a chessboard for camera calibration is easier than using a 3D rig because it's much simpler to print a 2D chessboard pattern on paper than to create an almost perfect 3D calibration rig.

When initially obtaining the B matrix, Cholesky factorization could not be performed directly. After checking, it was found that the eigenvalues were negative, so the entire B matrix was multiplied by -1 to ensure it became a positive definite matrix.

The formula of this method is derived based on the pinhole camera model, so distortion was not considered in our implementation. As a result, we could compute the intrinsic and extrinsic matrices using only linear algebra.

5. Conclusion

In this homework, we successfully implemented a camera calibration process from scratch. It's not only interesting to implement the 2D camera calibration, but also to learn the 2D image coordinate transformed to 3D world coordinate.

This approach, using a 2D chessboard for calibration, highlights the effectiveness of using a simple pattern to achieve precise results without relying on complex setups, for example a 3D rig.

Future improvements could involve extending this method to handle distortion.

6. Work Assignment Plan between Team Members

- 蔡濟謙
 - Code implementation
 - Implement Procedure part in this report
- 鄭博涵
 - Taking photos
 - Do experiments
 - Experimental Result part in this report
- 陳奎元
 - Do experiments
 - This report excluding Implement Procedure part

7. Command Line:

Teacher provided data:

```
python3 camera_calibration.py --mode data
```

Our smartphone data:

```
python3 camera_calibration.py --mode my_data
```


8. Output Explanation

- cv2_data : The extrinsic parameters visualization from the function provided by cv2 on the teacher provided data.
- cv2_my_data : The extrinsic parameters visualization from the function provided by cv2 on our own data.
- hw1_data : The extrinsic parameters visualization from our implementation on the teacher provided data.
- cv2_my_data : The extrinsic parameters visualization from our implementation on our own data.